

Reducción de Falsas Alarmas en la Detección Satelital de Incendios Forestales mediante Análisis Multivariable Acelerado por GPU sobre HPC

Informe Técnico Final

Allan Arnulfo Montes Monge, Esteban Gutiérrez Saborío, Maria Moroney Sole,

Marlon Mora Delgado y Robson Stiffen Calvo Ortega

Curso TTCT0017 – Computación Paralela y Distribuida

LEAD University – Profesor: Johansell Villalobos Cubillo

San José, Costa Rica

Repositorio: https://github.com/Allan1492/Deteccion_Incendios_Forestales_Imag_Satelitales

Abstract—Los incendios forestales constituyen una amenaza ambiental creciente cuyo monitoreo satelital genera volúmenes de datos que superan la capacidad de los enfoques secuenciales. Este trabajo presenta el desarrollo completo de un pipeline de cómputo paralelo y distribuido para la detección y el análisis de incendios forestales, ejecutado sobre el clúster de alto rendimiento Kabré del CeNAT. Se integran dos fuentes reales: 52 508 485 detecciones de puntos de calor de NASA FIRMS (sensor VIIRS) y 42 850 imágenes satelitales del *Wildfire Prediction Dataset*. El problema aplicado es la reducción de falsas alarmas en las detecciones. El proyecto abarca seis etapas: adquisición y gestión de datos, preprocesamiento paralelo, análisis exploratorio, modelado de aprendizaje automático y profundo, análisis de rendimiento y visualización interactiva. El preprocesamiento paralelo alcanza un *speedup* global de hasta $45\times$ frente a la línea base secuencial, con saturación en torno a los 16 hilos; los modelos tabulares (Random Forest, XGBoost, red neuronal, acelerados por GPU) logran un F1 de hasta 0.82 y un ROC-AUC de 0.98, y la clasificación de imágenes mediante *transfer learning* alcanza un F1 de 0.96 y un ROC-AUC de 0.99. El análisis CPU vs. GPU muestra aceleraciones de hasta $6.8\times$ y diferencias marcadas en la utilización efectiva del hardware. Se discuten los resultados a la luz de la ley de Amdahl y de los límites del paralelismo de memoria compartida, por procesos y por GPU.

Index Terms—incendios forestales, teledetección, cómputo paralelo, cómputo distribuido, HPC, GPU, RAPIDS, PyTorch, *speedup*, escalabilidad, Kabré.

I. INTRODUCCIÓN

Los incendios forestales han aumentado de forma sostenida en frecuencia, extensión y severidad durante las últimas décadas, impulsados por el aumento de las temperaturas, las sequías prolongadas y la acumulación de combustible vegetal. Sus consecuencias abarcan la pérdida de biodiversidad, la emisión de gases de efecto invernadero, el deterioro de la calidad del aire y daños a la infraestructura y a la vida humana. En este contexto, la detección temprana adquiere un valor estratégico.

La teledetección satelital ofrece una fuente privilegiada de información al observar de forma continua y a escala planetaria las anomalías térmicas de la superficie. Sin embargo, el volumen de datos generado por las misiones

modernas —millones de detecciones diarias y repositorios de imágenes de varios gigabytes— hace impráctico su procesamiento secuencial, lo que motiva el uso de técnicas de cómputo paralelo y distribuido.

La motivación de este proyecto es doble. Por un lado, abordar un problema ambiental relevante: la **reducción de falsas alarmas** en la detección de incendios, ya que no toda anomalía térmica corresponde a un incendio forestal real y los falsos positivos tienen un costo operativo. Por otro, y como eje del curso, aplicar y evaluar de forma cuantitativa técnicas de paralelismo multinúcleo, procesamiento distribuido y aceleración por GPU sobre una infraestructura de alto rendimiento real.

Este informe integra el trabajo completo del proyecto. La Sección II revisa el trabajo relacionado; la III, el marco teórico; la IV, la definición del problema; la V, los datos; la VI, la infraestructura; la VII, el diseño del pipeline; las Secciones IX–XIV, cada una de las seis etapas; la XV, la discusión; y la XVI concluye.

II. TRABAJO RELACIONADO

La detección operacional de incendios se basa en la firma térmica en el infrarrojo medio. El algoritmo de Colección 6 de MODIS [1] y el producto de fuego activo de 375 m de VIIRS [2] son la base de los productos que distribuye la NASA; la reducción de falsas alarmas (errores de comisión) ha sido uno de los principales motores de su evolución. En aprendizaje automático aplicado a la ciencia de incendios, Jain *et al.* [3] ofrecen una revisión exhaustiva que abarca detección, predicción de propagación y evaluación de riesgo. Para la clasificación de imágenes, las redes neuronales convolucionales, y en particular las redes residuales [4], son el enfoque dominante, frecuentemente mediante *transfer learning*.

En cómputo de datos a gran escala, Dask [5] se ha consolidado como marco de paralelización en Python; el ecosistema RAPIDS [7] lo extiende a GPU mediante cuDF y cuML; y Polars [6] aporta un motor columnar de alto

rendimiento sobre Apache Arrow. PyTorch [8] es el marco de referencia para aprendizaje profundo acelerado. Este proyecto se sitúa en la intersección de ambas líneas, con énfasis explícito en el diseño paralelo y la medición del rendimiento.

III. MARCO TEÓRICO

A. Modalidades de paralelismo

El cómputo paralelo reduce el tiempo de ejecución dividiendo un problema en subtarefas simultáneas. Se distinguen el *paralelismo multinúcleo* (varios núcleos con memoria compartida), el *procesamiento distribuido* (varios procesos o nodos que cooperan por paso de mensajes) y la *aceleración por GPU* (miles de hilos ligeros bajo un modelo SIMT). Cada modalidad tiene límites propios que este proyecto ilustra empíricamente.

B. Métricas de rendimiento

El *speedup* con p unidades de procesamiento se define como

$$S(p) = \frac{T_1}{T_p}, \quad (1)$$

donde T_1 es el tiempo secuencial y T_p el paralelo. La eficiencia es

$$E(p) = \frac{S(p)}{p}. \quad (2)$$

El límite teórico lo acota la ley de Amdahl [9], que para una fracción paralelizable f establece

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}}, \quad (3)$$

mientras que la ley de Gustafson [10] ofrece una perspectiva orientada al escalado del tamaño del problema, base de la *escalabilidad débil*. El análisis considera además la *escalabilidad fuerte* (tamaño fijo, recursos crecientes), el *throughput* (registros o imágenes por segundo), el uso de memoria y la utilización de GPU.

C. Aprendizaje automático y profundo

Para los datos tabulares se emplean clasificadores clásicos (Random Forest, *gradient boosting*) y una red neuronal; para las imágenes, una red convolucional mediante *transfer learning*. El entrenamiento es intensivo en cómputo y se beneficia de la aceleración por GPU, lo que lo convierte en un caso de estudio natural para el análisis de rendimiento.

IV. DEFINICIÓN DEL PROBLEMA

Los sistemas de detección satelital generan numerosas detecciones de puntos de calor, pero no todas corresponden a incendios forestales reales: fuentes de calor industriales, quemas agrícolas, reflejos o ruido del sensor producen **falsas alarmas** con un costo operativo real, pues desvían recursos de respuesta hacia eventos inexistentes. Aunque los productos operacionales alcanzan errores de comisión bajos —del orden del 1.2% para VIIRS y MODIS en píxeles de confianza nominal [2], [1]—, sobre decenas de millones de detecciones ese porcentaje se traduce en cientos de miles de falsos

positivos, y la fracción de detecciones de baja confianza es sustancialmente mayor. El problema que se plantea es: *a partir de la combinación de dos o más variables físicas y contextuales de cada detección, ¿es posible discriminar las detecciones confiables de las que probablemente son falsas alarmas, reduciendo la tasa de falsos positivos?* En lugar de confiar en un único umbral —como haría una regla simple—, se propone combinar varias señales para tomar una mejor decisión.

A. Definición operativa

El conjunto de FIRMS presenta una limitación clave: contiene únicamente detecciones y ninguna viene marcada como “verdadera” o “falsa”; no existe una verdad de campo. Para abordar el problema de forma medible se adopta una definición operativa basada en el atributo *confidence* (nivel de confianza que el propio sistema asigna a cada detección): las detecciones de confianza **media o alta** se tratan como **válidas**, y las de confianza **baja** como **probables falsas alarmas**. Con esto, el problema se formaliza como una tarea de **clasificación binaria** bien definida. Es fundamental que la variable *confidence* se use únicamente para construir la etiqueta objetivo y **nunca como variable de entrada**, de modo que el modelo aprenda de las variables físicas y no “copie” la respuesta. En el conjunto analizado, la clase de baja confianza representa el 12.15% de las detecciones (Sección XI). La hipótesis central es que el contraste térmico $\Delta T = \text{brightness} - \text{bright_t31}$, combinado con la potencia radiativa, permite discriminar las clases; la Sección XI la contrasta empíricamente.

V. DATOS

El proyecto integra dos fuentes reales y complementarias (Tabla I).

NASA FIRMS (rama tabular). Puntos de calor del sensor VIIRS (satélites Suomi-NPP, NOAA-20 y NOAA-21), del 30 de junio de 2025 al 30 de junio de 2026: 52 508 485 registros, 14 columnas, 1.24 GB en Apache Parquet, sin valores nulos. Las columnas *brightness* y *bright_t31* conservan la nomenclatura de MODIS pero corresponden a las bandas VIIRS I-4 (3.74 μm , canal del fuego) e I-5 (11.45 μm , fondo térmico); su diferencia, ΔT , mantiene el mismo significado físico.

Kaggle Wildfire Prediction Dataset (rama de imágenes). 42 850 imágenes satelitales de 350×350 píxeles etiquetadas en *wildfire/nowildfire*, aproximadamente 6.5 GB.

VI. INFRAESTRUCTURA DE CÓMPUTO

Los experimentos se ejecutaron en el clúster **Kabré** del Centro Nacional de Alta Tecnología (CeNAT), gestionado con SLURM. Se emplearon la partición *kura* (40 núcleos, 257 GB de RAM) para el procesamiento en CPU y las particiones *nukwa* con GPUs NVIDIA (V100 y L40S) para el modelado acelerado. El entorno de CPU se construyó con Miniforge (Python 3.11, Polars, Dask); el de GPU con el entorno *rapids* del clúster y PyTorch con CUDA. Todo el cómputo

TABLE I
CONJUNTOS DE DATOS DEL PROYECTO

Atributo	NASA FIRMS	Kaggle Wildfire
Tipo	Tabular (puntos de calor)	Imágenes satelitales
Sensor/Fuente	VIIRS (SNPP, N20, N21)	MapBox / Quebec
Volumen	52 508 485 registros	42 850 imágenes
Tamaño	1.24 GB (Parquet)	≈6.5 GB
Uso	ML tabular + EDA	CNN (aprendizaje profundo)

TABLE II
CONFIGURACIÓN EXPERIMENTAL

Componente	Detalle
Clúster	Kabré (CeNAT), gestor SLURM
Nodo CPU	Partición kura: 40 núcleos, 257 GB RAM
Nodo GPU	Partición nukwa: GPU NVIDIA (V100/L40S)
Lenguaje	Python 3.11
Preprocesamiento	pandas, Polars, Dask (Miniforge)
Modelado	RAPIDS cuML, XGBoost, PyTorch (CUDA)
Visualización	Plotly, Dash
Repeticiones	3 por config. (mediana); <i>hold-out</i> para modelos
Escalas	30 %, 60 % y 100 % del dataset (GPU)

intensivo se despachó mediante `sbatch/srun` reservando nodos completos, para no contaminar las mediciones con la carga de otros usuarios. Los datos masivos se almacenaron en el sistema de archivos `/data` (100 GB de cuota), no en `/home` (10 GB).

A. Configuración experimental

Para garantizar la reproducibilidad, la Tabla II resume la configuración experimental. El benchmark de preprocesamiento se ejecutó en la partición `kura` (memoria compartida), reservando un nodo completo de 40 núcleos y fijando las variables `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS` y `MKL_NUM_THREADS` en 1 para impedir que las bibliotecas de álgebra lineal generaran hilos adicionales que contaminaran la medición del paralelismo. El modelado y el benchmark CPU vs. GPU se ejecutaron en la partición `nukwa` con GPU NVIDIA. Cada configuración del benchmark de CPU se repitió tres veces, reportándose la mediana; los modelos se evaluaron sobre una partición de prueba independiente (*hold-out*).

VII. DISEÑO DEL PIPELINE

El sistema se organiza en seis etapas (Fig. 1). La rama tabular (FIRMS) y la rama de imágenes (Kaggle) se procesan en paralelo y son **independientes**: no se combinan en un único modelo, sino que resuelven dos tareas complementarias bajo el mismo marco de cómputo paralelo, convergiendo en el análisis de resultados y la visualización.

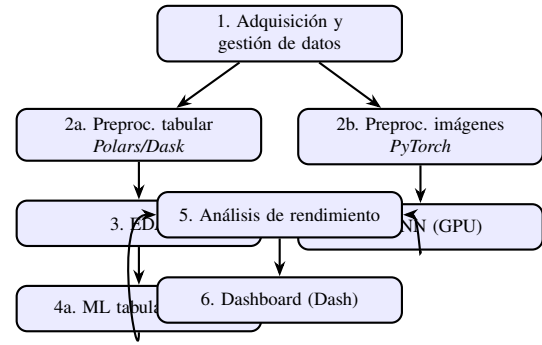


Fig. 1. Pipeline de seis etapas con dos ramas paralelas e independientes.

TABLE III
CRONOGRAMA DEL PROYECTO (SEMANAS 8 A 14)

Sem.	Fase / Entrega	Etapas
8	Adquisición y gestión de datos	Etapas 1
9	Preprocesamiento paralelo/distrib.	Etapas 2
10	EDA + Entrega 2 (30 %)	Etapas 3
11	Modelado ML/IA	Etapas 4
12	Optimización y postprocesamiento	Etapas 4–5
13	Rendimiento, visualización, informe	Etapas 5–6
14	Entrega 3 (40 %) + Entrega 4 (10 %)	—

VIII. PLAN DE TRABAJO Y ORGANIZACIÓN

El proyecto se organizó en cuatro entregas incrementales (con pesos de 20 %, 30 %, 40 % y 10 %) desarrolladas entre las semanas 8 y 14 del cuatrimestre, según el cronograma de la Tabla III. El trabajo se repartió de forma equitativa entre los cinco integrantes, cada uno responsable principal de una etapa del pipeline: adquisición y gestión de datos junto con la administración del repositorio (E. Gutiérrez); preprocesamiento paralelo y distribuido (A. Montes); análisis exploratorio y estadístico (M. Moroney); modelado de aprendizaje automático y profundo (M. Mora); y postprocesamiento, análisis de rendimiento y visualización (R. Calvo). La redacción del informe, la administración del repositorio y la presentación se coordinaron de forma compartida.

IX. ETAPA 1: ADQUISICIÓN Y GESTIÓN DE DATOS

Los registros de FIRMS se descargaron de la fuente y se consolidaron en un único archivo Parquet, formato elegido por su compresión eficiente, su lectura selectiva por columnas y su compatibilidad nativa con los motores paralelos del proyecto. Durante el desarrollo se detectó que una copia del archivo estaba **truncada**, lo que provocó que el análisis se ejecutara, sin aviso, sobre datos sintéticos de respaldo. Para evitar la repetición del problema se incorporó al pipeline una rutina de verificación de integridad que comprueba los bytes mágicos `PAR1`, valida los metadatos, realiza una lectura de prueba y confirma la presencia de las columnas requeridas antes de cada trabajo de cómputo. Dado que las cuentas del clúster son

personales, los datos se compartieron entre el equipo mediante almacenamiento en la nube, cargándose luego a /data en cada cuenta.

X. ETAPA 2: PREPROCESAMIENTO PARALELO Y RENDIMIENTO

A. Preprocesamiento y variables derivadas

La rama tabular se limpió y transformó con Polars y Dask. Se calcularon las variables del modelo: el contraste térmico ΔT , el mes (de la fecha), la hora (de `acq_time`), un indicador de noche y la variable objetivo `es_falsa_alarma`. Se descartaron registros con potencia radiativa negativa o coordenadas fuera de rango. El resultado se escribió en *streaming* (`sink_parquet`), procesando los 52.5 millones de registros por bloques sin materializarlos en memoria.

B. Diseño del benchmark

Para evaluar la ganancia del paralelismo se comparó una línea base **estrictamente secuencial** (pandas restringido a un hilo) contra Polars (memoria compartida, 1–40 hilos) y Dask (procesos distribuidos), sobre cuatro operaciones representativas: carga, filtrado, agregación y cálculo de características. La elección de pandas como base es deliberada: el motor *in-memory* de Polars también es multihilo, por lo que no serviría como referencia secuencial.

Se distinguen dos aceleraciones: la **global** (T_{pandas}/T_p), que mide la ganancia total de cambiar de motor y paralelizar; y la **paralela** ($T_{\text{motor}}(1)/T_{\text{motor}}(p)$), que aísla el efecto del paralelismo. La eficiencia se calcula con la segunda, pues usar la global produciría eficiencias superiores al 100%. Cada configuración se repitió tres veces, reportándose la mediana.

C. Resultados

La Tabla IV y la Fig. 2 resumen los resultados de la agregación. Se alcanza un **speedup global de hasta 45×**, descompuesto en $4.97\times$ por el cambio de motor y $9.14\times$ por el paralelismo. El sistema **satura en torno a los 16 hilos**: más allá, el tiempo deja de mejorar y la eficiencia cae de 57% (16 hilos) a 20% (40 hilos), como muestra la Fig. 3. Este comportamiento es consistente con la ley de Amdahl (Ec. 3): la fracción no paralelizable (lectura de disco, coordinación) y la contención por el ancho de banda de memoria imponen un techo. El *throughput* de Polars alcanza un máximo de 141 millones de filas por segundo hacia los 16 hilos, confirmando la misma tendencia de saturación.

Un segundo hallazgo es que **Dask falla por memoria** en las operaciones intensivas al superar los 8 *workers*: al ser paralelismo por procesos, cada *worker* replica su porción de datos y el consumo total agota la memoria del nodo, límite ausente en el modelo de hilos de Polars. En el experimento de escalabilidad débil, en cambio, el *throughput* de Polars se mantiene elevado y estable al crecer la carga en proporción a los hilos.

TABLE IV
RENDIMIENTO DEL PREPROCESAMIENTO (AGREGACIÓN, 52.5 M REGISTROS)

Motor	p	T (s)	S_{par}	E (%)
pandas	1	11.60	—	—
Polars	1	2.33	1.00	100
Polars	4	0.613	3.80	95
Polars	8	0.354	6.59	82
Polars	16	0.255	9.14	57
Polars	32	0.275	8.48	27
Polars	40	0.285	8.18	20
Dask	4	33.9	1.82	45
Dask	≥ 8	fallo por memoria (<i>workers</i> terminados)		

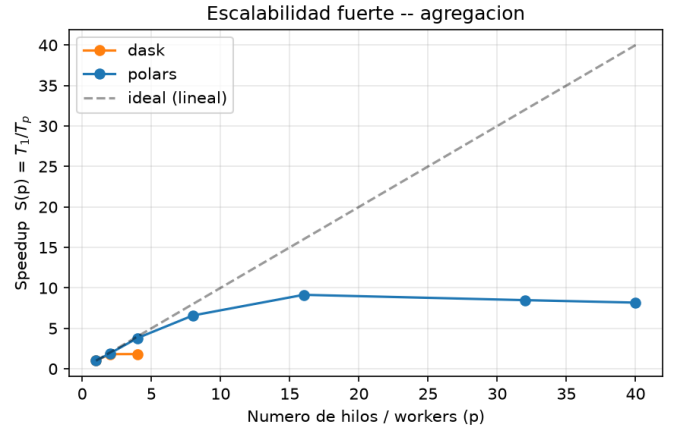


Fig. 2. Escalabilidad fuerte (speedup paralelo) de la agregación. La curva de Polars se aparta de la recta ideal y satura hacia los 16 hilos.

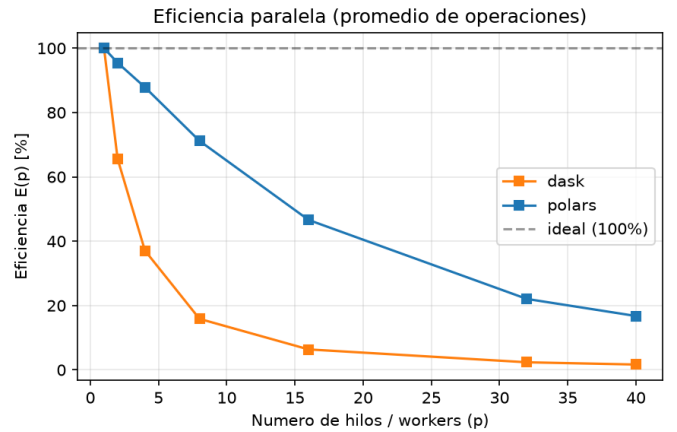


Fig. 3. Eficiencia paralela frente al número de hilos. La caída sostenida refleja el efecto descrito por la ley de Amdahl.

XI. ETAPA 3: ANÁLISIS EXPLORATORIO

El EDA se ejecutó con Polars en modo *streaming* sobre los datos completos.

TABLE V
CONTRASTE TÉRMICO Y FRP MEDIOS POR NIVEL DE CONFIANZA

Confianza	ΔT (K)	FRP (MW)	Registros
Alta	60.81	28.21	2 912 108
Nominal	34.78	8.41	43 214 217
Baja	28.11	12.79	6 382 160

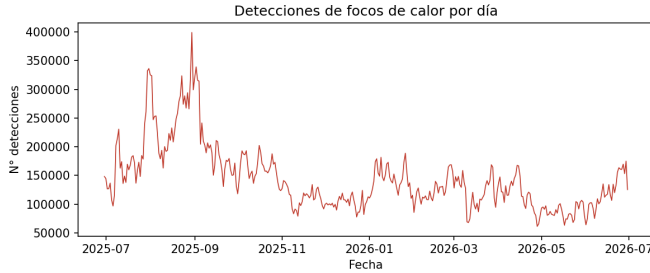


Fig. 4. Serie temporal diaria de detecciones. El pico de finales de agosto corresponde a la temporada de incendios boreal.

A. Distribución temporal y espacial

La serie temporal diaria (Fig. 4) muestra una marcada estacionalidad, con un máximo de 399 153 detecciones el 30 de agosto de 2025 y un mínimo invernal, coherente con la temporada de incendios boreal. El análisis mensual y horario confirma el patrón estacional y la mayor detectabilidad en determinadas franjas. Espacialmente, las mayores densidades se concentran en el centro de Canadá, Siberia y el África subsahariana; los tres satélites VIIRS aportan volúmenes casi idénticos.

B. Variable objetivo e hipótesis

La distribución de la variable objetivo (Fig. 5) presenta un desbalance aproximado de 7:1: 87.85 % de detecciones válidas frente a 12.15 % de baja confianza. Este desbalance implica que la exactitud no es una métrica adecuada, por lo que la evaluación se basa en precisión, exhaustividad, F1 y matriz de confusión.

El hallazgo más relevante valida la hipótesis del proyecto (Tabla V): el contraste térmico ΔT discrimina las clases. Las detecciones de alta confianza presentan un ΔT medio de 60.8 K, más del doble que las de baja confianza (28.1 K). Los focos que apenas se distinguen de su entorno térmico tienden a clasificarse con baja confianza, confirmando que las variables físicas contienen señal discriminativa. La distribución del FRP (Fig. 6) está fuertemente sesgada a la derecha, típico de fenómenos de incendio.

XII. ETAPA 4: MODELADO

A. Rama tabular: reducción de falsas alarmas

Sobre el conjunto preprocesado (11 predictoras, variable objetivo `es_falsa_alarma`) se entrenaron tres modelos, todos acelerados por GPU: **Random Forest** (RAPIDS cuML), **XGBoost** y una **red neuronal** (PyTorch). Previamente se

Nivel de confianza de las detecciones

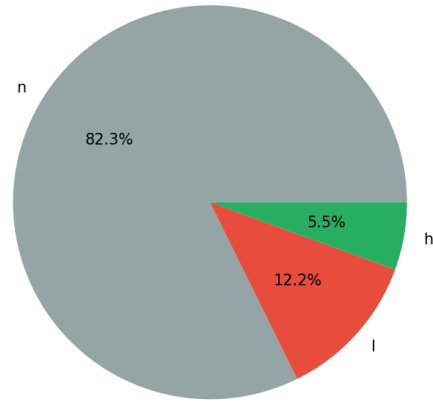


Fig. 5. Distribución del nivel de confianza (variable objetivo). Predomina la clase nominal; la baja confianza (*proxy* de falsa alarma) es el 12.15 %.

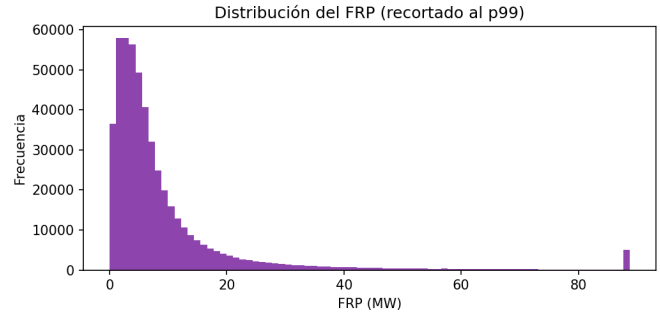


Fig. 6. Distribución de la potencia radiativa del fuego (FRP), fuertemente sesgada a la derecha.

TABLE VI
HIPERPARÁMETROS SELECCIONADOS

Modelo	Configuración
Random Forest	<code>n_estimators=100, max_depth=16</code>
XGBoost	<code>max_depth=4, eta=0.3</code>
Red Neuronal	<code>batch_size=2048, lr=0.001</code>

realizó una búsqueda de hiperparámetros (Fig. ??) cuyo criterio de selección no fue únicamente el F1, sino, entre las configuraciones dentro de 0.01 del mejor F1, la más rápida de entrenar; los valores elegidos se listan en la Tabla VI.

La Tabla VII y las Figs. 7–9 resumen las métricas de calidad. Los tres modelos alcanzan un ROC-AUC cercano a 0.98, confirmando la capacidad discriminativa anticipada por el EDA. La red neuronal obtiene el mejor F1 (0.816); XGBoost destaca por su altísimo *recall* (0.981), deseable cuando el costo de omitir una falsa alarma es alto. La elección operativa depende del compromiso entre precisión y exhaustividad.

TABLE VII
MODELOS TABULARES (GPU, DATASET REAL, HIPERPARÁMETROS AJUSTADOS)

Modelo	Precisión	Recall	F1	ROC-AUC
Random Forest	0.755	0.859	0.803	0.980
XGBoost	0.678	0.981	0.802	0.983
Red Neuronal	0.746	0.901	0.816	0.982

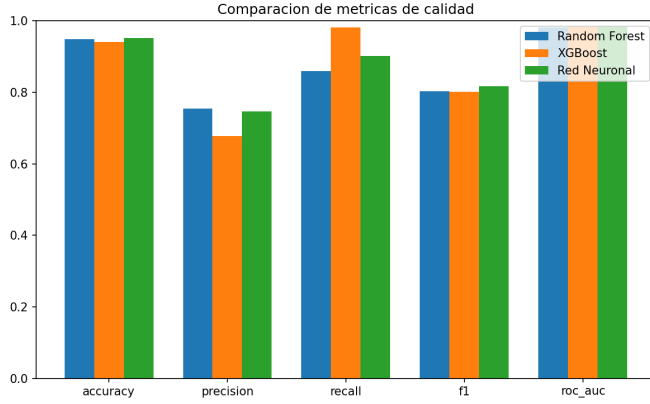


Fig. 7. Comparación de métricas de calidad entre los tres modelos tabulares.

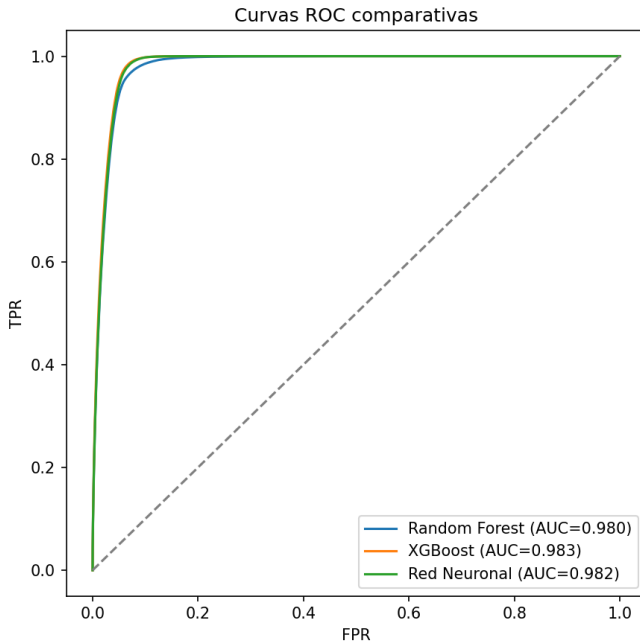


Fig. 8. Curvas ROC de los modelos tabulares; todas con AUC $\approx$ 0.98.

B. Rama de imágenes: clasificación con CNN

La clasificación de imágenes se abordó con *transfer learning* sobre ResNet18 preentrenada en ImageNet, congelando las capas convolucionales y entrenando solo una nueva capa clasificadora (aproximadamente el 0.3% de los parámetros), lo que reduce drásticamente el tiempo

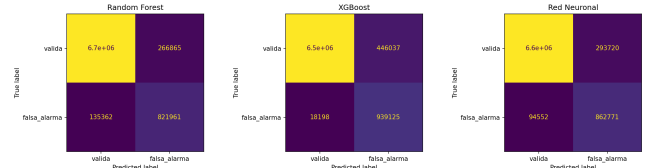


Fig. 9. Matrices de confusión de los modelos tabulares.

TABLE VIII
CLASIFICACIÓN DE IMÁGENES (CNN, RESNET18, GPU)

Accuracy	Precisión	Recall	F1	ROC-AUC
0.961	0.973	0.955	0.964	0.992

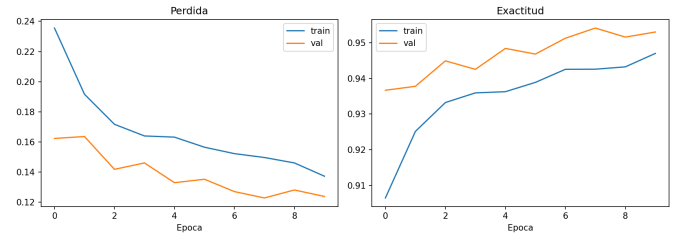


Fig. 10. Curvas de entrenamiento de la CNN (pérdida y exactitud por época).

de entrenamiento. Sobre 30250 imágenes de entrenamiento y 6300 de prueba, el modelo alcanza los resultados de la Tabla VIII, con las curvas de entrenamiento y la matriz de confusión de las Figs. 10–11. El dataset incluía imágenes truncadas que se toleraron habilitando la carga de imágenes incompletas.

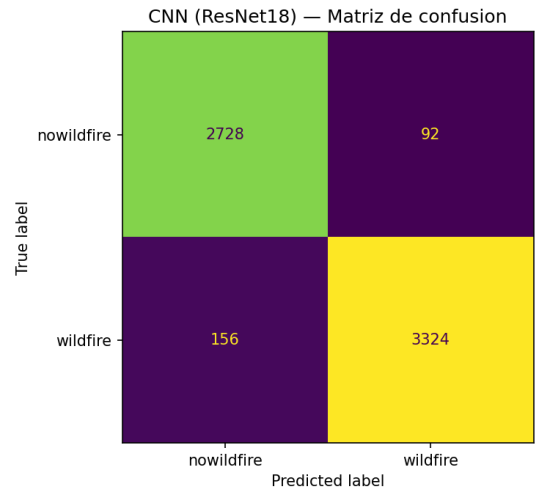


Fig. 11. Matriz de confusión de la CNN sobre el conjunto de prueba.

TABLE IX
ACELERACIÓN POR GPU (100% DEL DATASET, ≈ 36.8 M REGISTROS DE ENTRENAMIENTO)

Modelo	CPU (s)	GPU (s)	Speedup	GPU util.
Random Forest	1582.6	240.8	6.6 $\times$	59 %
XGBoost	115.3	16.9	6.8 $\times$	96 %
Red Neuronal*	15.2	7.8	1.9 $\times$	38 %

*Tiempo por época.

XIII. ETAPA 5: ANÁLISIS DE RENDIMIENTO CPU VS. GPU

El modelado se instrumentó para medir el rendimiento entrenando cada algoritmo en CPU y en GPU sobre los mismos datos, a tres escalas (30 %, 60 % y 100 % del dataset). La Tabla IX y la Fig. 12 reportan los resultados a escala completa; la Fig. 13 detalla el uso de recursos.

El hallazgo más valioso no es la aceleración en sí, sino que **no toda aceleración por GPU es igual de eficiente**. XGBoost aprovecha la GPU al 96 %, mientras que la red neuronal apenas al 38 % y Random Forest al 59 % (Fig. 13). La baja utilización de la red neuronal se explica porque, con estos tamaños de lote, el cuello de botella no es el cómputo sino la alimentación de datos (I/O y transformación en CPU). De hecho, en la CNN de imágenes, aumentar el tamaño de lote y el número de trabajadores de carga redujo el tiempo de 302.6 s a 92.1 s (3.3 $\times$) sin pérdida de calidad, ilustrando que el rendimiento depende tanto del algoritmo como del flujo de datos que lo alimenta.

Se reporta con transparencia una anomalía observada: el Random Forest en CPU presentó tiempos casi idénticos a 60 % (1495 s) y a 100 % (1583 s) del dataset, en lugar de crecer proporcionalmente. La causa probable es que esas corridas se ejecutaron en sesiones distintas del clúster, con condiciones de carga del nodo diferentes; se documenta para no inducir conclusiones erróneas sobre la escalabilidad de ese algoritmo en CPU.

A. Cuellos de botella, balance de carga y comunicación

El análisis permite identificar los principales cuellos de botella del sistema. En el preprocesamiento, el cuello es el **ancho de banda de memoria** y la fracción secuencial de E/S, que provocan la saturación a partir de los 16 hilos (Sección X). En el modelado sobre GPU, el cuello se desplaza a la **alimentación de datos**: cuando el lote es pequeño, la carga y transformación en CPU no logran mantener ocupada la GPU, como evidencia la baja utilización de la red neuronal (38 %).

Respecto al **balance de carga**, el modelo de hilos de Polars reparte el trabajo de forma homogénea entre los núcleos mediante su planificador interno, y en Dask el planificador distribuye las particiones entre los *workers*; el desbalance solo aparece cuando el número de particiones no es múltiplo del número de *workers*, lo que se mitigó ajustando el particionado. En cuanto a los **costos de comunicación**, son precisamente

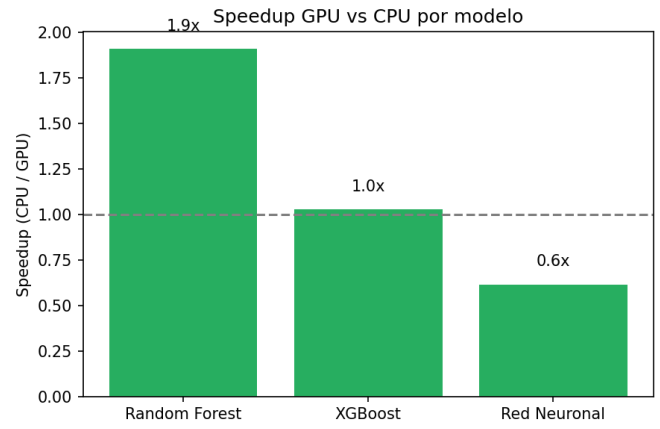


Fig. 12. Speedup GPU vs. CPU por modelo.

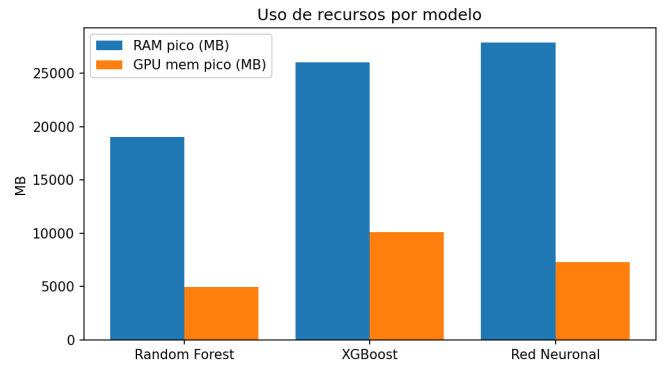


Fig. 13. Uso de recursos: RAM pico, memoria y utilización de GPU por modelo.

el factor que penaliza a Dask frente a Polars en un solo nodo: al ejecutarse como procesos independientes, Dask debe serializar y transferir datos entre *workers* (comunicación entre procesos), un costo ausente en el modelo de memoria compartida de Polars, que comparte un único espacio de direcciones. Este sobre costo de comunicación solo se amortiza cuando el volumen de datos o el número de nodos es suficientemente grande, escenario de la ejecución distribuida multinodo propuesta como trabajo futuro.

Como **estrategias de optimización** se aplicaron: el uso de *streaming* para evitar materializar los datos en memoria; el ajuste del tamaño de lote y del número de trabajadores de carga en la CNN (que redujo el tiempo 3.3 $\times$); y la selección de hiperparámetros ponderando calidad y tiempo de entrenamiento.

XIV. ETAPA 6: VISUALIZACIÓN INTERACTIVA

Los resultados de rendimiento se integraron en un *dashboard* web interactivo construido con Plotly y Dash. La herramienta permite explorar, con filtros por modelo y por escala de datos, los tiempos de cómputo, el *speedup* CPU vs. GPU, la utilización de recursos y las métricas de calidad,

sirviendo como interfaz de comunicación de los hallazgos del proyecto y como demostración de apoyo en la presentación.

XV. DISCUSIÓN

El proyecto ilustra tres modalidades de paralelismo y sus límites característicos. El **paralelismo de memoria compartida** (Polars) ofrece la mejor relación rendimiento/complejidad para el preprocesamiento, pero satura cuando la fracción secuencial y el ancho de banda de memoria dominan (ley de Amdahl). El **paralelismo por procesos** (Dask) escala más allá de una máquina, pero paga un costo de memoria y coordinación que, en un solo nodo, lo hace fallar en operaciones intensivas. La **aceleración por GPU** (RAPIDS, PyTorch) brinda las mayores ganancias en el modelado, pero su eficiencia depende críticamente de mantener ocupado el dispositivo, lo que a su vez depende del algoritmo y del flujo de datos. En conjunto, el trabajo confirma empíricamente que la ganancia del cómputo paralelo es real y cuantificable, pero está siempre acotada por la parte no paralelizable del problema y por la arquitectura de memoria.

XVI. CONCLUSIÓN Y TRABAJO FUTURO

Se desarrolló un pipeline completo de cómputo paralelo y distribuido para la detección de incendios forestales sobre datos reales y a escala (52.5 millones de registros y 42 850 imágenes), ejecutado en el clúster Kabré. El preprocesamiento paralelo alcanzó un *speedup* global de hasta 45×; los modelos tabulares lograron un ROC-AUC de 0.98 en la reducción de falsas alarmas, y la clasificación de imágenes un F1 de 0.96 y un ROC-AUC de 0.99. El análisis CPU vs. GPU evidenció aceleraciones de hasta 6.8× y, sobre todo, diferencias sustanciales en la utilización efectiva del hardware entre algoritmos. Los resultados validan el diseño propuesto y, más allá del dominio de aplicación, constituyen un estudio concreto de los límites del paralelismo de memoria compartida, por procesos y por GPU.

A. Lecciones aprendidas

El proyecto dejó varias lecciones de valor práctico. Primero, la **integridad de los datos** no es trivial a escala: un archivo Parquet truncado provocó que el análisis corriera sin aviso sobre datos sintéticos de respaldo, lo que obligó a incorporar una verificación de integridad como paso obligatorio del pipeline. Segundo, **más paralelismo no siempre es mejor**: el punto de saturación en 16 hilos y el fallo por memoria de Dask muestran que la arquitectura de memoria impone límites que deben medirse, no asumirse. Tercero, en cómputo por GPU, **el cuello de botella no siempre es el modelo**: con frecuencia es el flujo de datos que lo alimenta, y optimizarlo (tamaño de lote, trabajadores de carga) rindió aceleraciones mayores que cambiar el modelo. Cuarto, la **metodología de medición importa**: distinguir la aceleración por cambio de motor de la aceleración por paralelismo evitó reportar eficiencias imposibles. Estas lecciones son transferibles a cualquier proyecto de datos a gran escala.

B. Trabajo futuro

Como trabajo futuro se plantea la ejecución multinodo distribuida con Dask, la optimización del flujo de datos hacia la GPU para elevar su utilización, la incorporación de cuantificación de incertidumbre en las predicciones, y la integración de ambas ramas en un sistema de alerta operacional en tiempo casi real.

REFERENCES

- [1] L. Giglio, W. Schroeder, y C. O. Justice, "The collection 6 MODIS active fire detection algorithm and fire products," *Remote Sensing of Environment*, vol. 178, pp. 31–41, 2016.
- [2] W. Schroeder, P. Oliva, L. Giglio, y I. A. Csiszar, "The New VIIRS 375 m active fire detection data product," *Remote Sensing of Environment*, vol. 143, pp. 85–96, 2014.
- [3] P. Jain *et al.*, "A review of machine learning applications in wildfire science and management," *Environmental Reviews*, vol. 28, no. 4, pp. 478–505, 2020.
- [4] K. He, X. Zhang, S. Ren, y J. Sun, "Deep residual learning for image recognition," en *Proc. IEEE CVPR*, 2016, pp. 770–778.
- [5] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," en *Proc. 14th Python in Science Conf.*, 2015, pp. 130–136.
- [6] R. Vink y col., "Polars: Lightning-fast DataFrame library for Rust and Python." [En línea]. Disponible: <https://pola.rs/>
- [7] RAPIDS Development Team, "RAPIDS: Libraries for end-to-end GPU data science," NVIDIA. [En línea]. Disponible: <https://rapids.ai/>
- [8] A. Paszke y col., "PyTorch: An imperative style, high-performance deep learning library," en *NeurIPS*, vol. 32, 2019, pp. 8024–8035.
- [9] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," en *Proc. AFIPS*, 1967, pp. 483–485.
- [10] J. L. Gustafson, "Reevaluating Amdahl's law," *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [11] Colaboratorio Nacional de Computación Avanzada, "Kabré: infraestructura de supercomputación del CeNAT." [En línea]. Disponible: <https://kabre.cenat.ac.cr/>
- [12] NASA, "Fire Information for Resource Management System (FIRMS)." [En línea]. Disponible: <https://firms.modaps.eosdis.nasa.gov/>